01 QUESTION ONE · DBMS

What do you mean by DBMS? Explain the purpose of DBMS.

4 A4 PAGES
SQL PROGRAM ON PAGE 4

WHAT IS A DBMS?

A **DBMS (Database Management System)** is a computer program that stores, retrieves and manages data in a structured way. It provides an **interface** between the user and the raw database, and it is responsible for keeping the data **secure**, **consistent** and **free from redundancy**.

Suppose a college stores student records. Without a DBMS every clerk keeps his own list, so the same student appears in several files with different marks. With a DBMS the data lives in **one** place, every user sees the same copy, and the program checks every request before it is allowed to touch a record.

The main functions of a DBMS are storing and retrieving data, defining the structure of the tables, enforcing security, allowing several users to work at the same time, and taking a **backup** so that data is never lost.

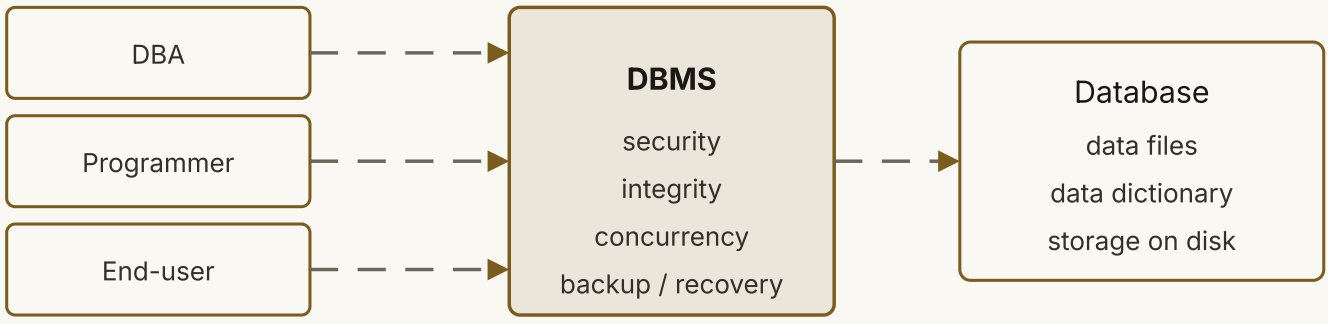


FIG. 1 — A DBMS DOES NOT SIT BESIDE THE DATA; IT STANDS **BETWEEN** THE USERS AND THE DATA, CHECKING EVERY REQUEST BEFORE IT IS ALLOWED TO TOUCH THE RECORDS.

PURPOSE OF DBMS

1. To remove data redundancy. The same fact is stored only once, so a change is made in one place and the whole system sees it.

2. To remove data inconsistency. Because there is a single copy, two clerks can never disagree about a student's marks.

3. To share the data. Several users and programs may read and write the same database at the same time.

4. To keep the data secure. A password and permission list decide who may see which table, and every attempt is checked.

5. To allow independence of data. The physical storage can be changed — from one disk to another — without changing any program, because the DBMS hides the storage details.

6. To take a backup. The DBMS can copy the whole database to another device, so the data survives a crash, a theft or a fault.

7. To provide an easy interface. The user does not have to understand where the bytes are kept; he only has to ask in SQL.

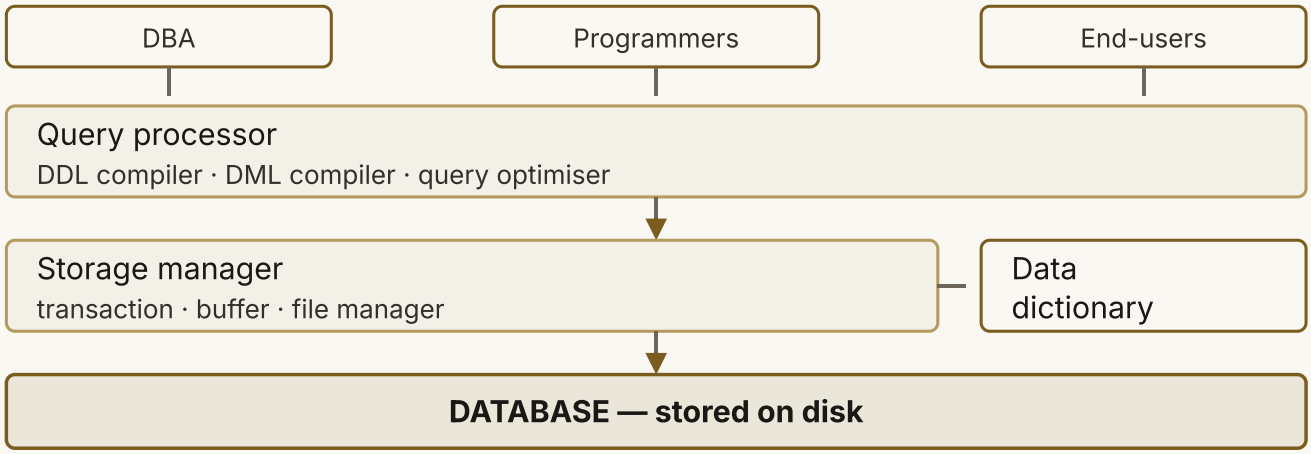
8. To keep standards. One common format for data makes it easy to exchange information between different departments and programmes.

9. To handle large data. Using an **index**, a small extra structure that lets the DBMS jump straight to the rows it needs, a question over millions of records is answered in seconds. **10. To enforce rules.** A rule such as "this roll number can never repeat" is written once into the table, and the DBMS refuses any entry that breaks it, whatever program sends it.

COMPONENTS OF A DBMS

A DBMS works with the help of five components. **Hardware** is the computers and storage devices on which the data is kept. **Software** is the DBMS program itself, which stands between the users and the database. **Data** is the actual information stored in the database. **Procedures** are the rules that tell how the DBMS is to be used and managed. **Users** are the people who work with the system — the database administrator, the application programmers and the ordinary end-users.

Inside the DBMS sits the **Data Dictionary**, also called the system catalog. It stores **metadata** — data about data: every table name, column name, data type, constraint and user permission. Whenever a query is run, the DBMS first looks in the data dictionary to understand the shape of the table, and only then fetches the data.



EVERY REQUEST IS CHECKED BEFORE IT IS ALLOWED TO TOUCH THE DATA

FIG. 2 — THE LAYERS OF A DBMS. USERS SEND A REQUEST TO THE QUERY PROCESSOR, THE QUERY PROCESSOR ASKS THE STORAGE MANAGER FOR THE BYTES, AND THE DATA DICTIONARY IS CONSULTED AT EVERY STEP SO THAT THE SYSTEM ALWAYS KNOWS THE SHAPE OF ITS OWN TABLES.

SQL PROGRAM

A SMALL PROGRAM THAT CREATES A TABLE, INSERTS ROWS AND READS THEM BACK.

DBMS_DEMO.SQL

```sql
-- create the table (DDL)
CREATE TABLE STUDENT (
  ROLL_NO   NUMBER(4) PRIMARY KEY,
  NAME      VARCHAR2(20) NOT NULL,
  CITY      VARCHAR2(15),
  FEE       NUMBER(8)
);

-- insert the rows (DML)
INSERT INTO STUDENT VALUES (101,'Rahul','Patna',4500);
INSERT INTO STUDENT VALUES (102,'Anjali','Patna',5100);
INSERT INTO STUDENT VALUES (103,'Vikas','Gaya', 4800);

-- read the data back
SELECT ROLL_NO, NAME, FEE FROM STUDENT WHERE FEE > 4600;

-- change and delete
UPDATE STUDENT SET FEE = 5000 WHERE ROLL_NO = 103;
DELETE FROM STUDENT WHERE ROLL_NO = 102;
```

Output

OUTPUT

```
ROLL_NO  NAME    FEE
-------  ------  -----
102      Anjali  5100
103      Vikas   4800
```

CONCLUSION

A DBMS keeps data in one place, checks every request and removes the problems of ordinary files.

02 QUESTION TWO · NORMALIZATION

Explain Normalization with Example.

4 A4 PAGES
SQL EXAMPLE ON PAGE 4

WHAT IS NORMALIZATION?

Normalization is the process of organising the columns and tables of a relational database so that each fact is stored exactly once. A table is said to be in a **normal form** when it obeys a stated rule.

Normalization removes three kinds of defect that make a database behave badly: **update anomaly**, where one change has to be repeated in several rows; **insert anomaly**, where a row cannot be added because some information is not yet known; and **delete anomaly**, where deleting one fact accidentally removes another.

The three forms are studied in order, and each removes a different defect: **1NF** deals with a cell holding several values, **2NF** with a column that depends on only half the key, and **3NF** with a column that depends on another non-key column. Normalisation is judged by reading the table, not by a formula, and a table reaches the next form only after obeying the earlier one.

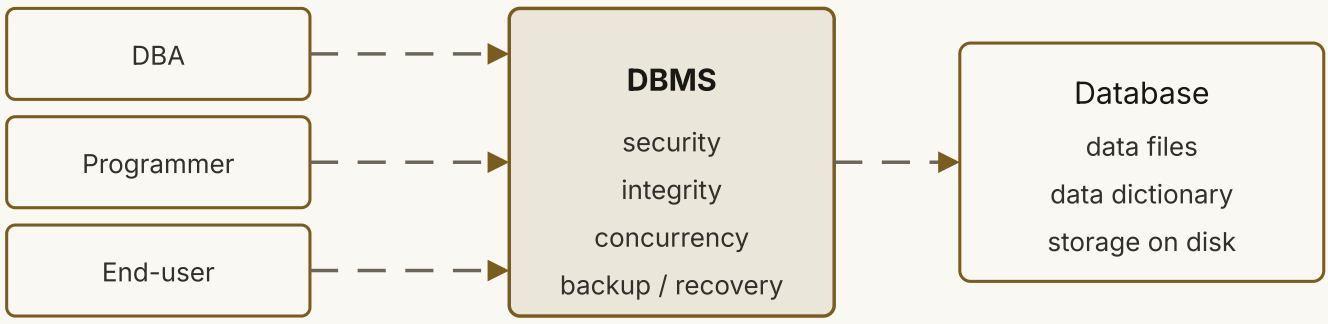


FIG. 1 — A DBMS DOES NOT SIT BESIDE THE DATA; IT STANDS **BETWEEN** THE USERS AND THE DATA, CHECKING EVERY REQUEST BEFORE IT IS ALLOWED TO TOUCH THE RECORDS.

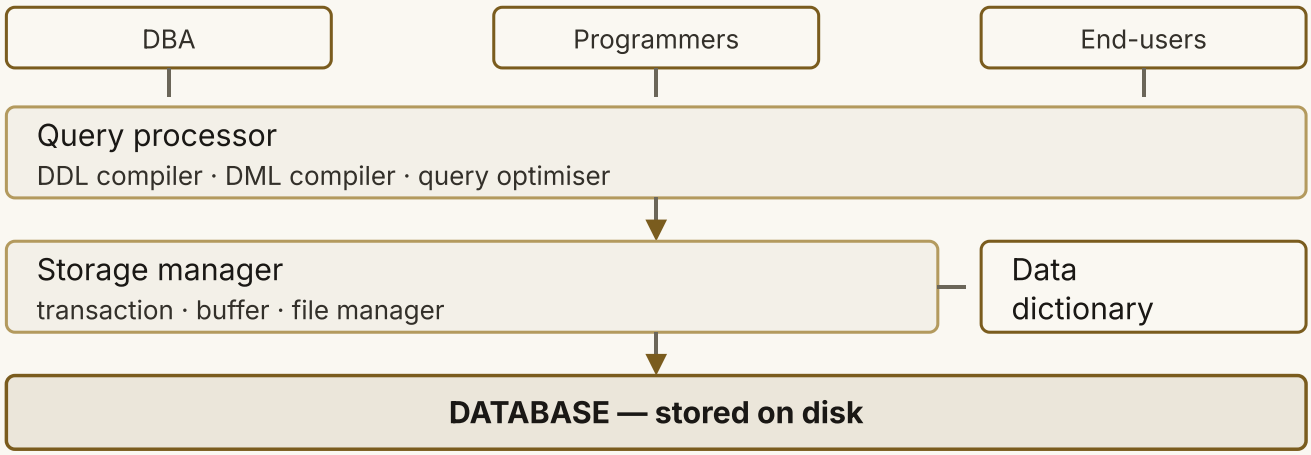
FIRST NORMAL FORM (1NF)

A table is in **1NF** when every cell contains a **single atomic value** — one indivisible item — and there is no repeating group of columns.

Suppose a student is stored with his courses in one cell. That cell holds “BCA, BBA”, which is really two values, so the table breaks 1NF.

The cure is simple: break the cell open and give each value a row of its own. The two courses of one student then occupy two rows, both carrying the same roll number, so the student is still counted once. This is shown in the figure below.

A set of columns that all repeat together — `SUB1`, `SUB2`, `SUB3` and so on, one pair for every possible subject — is a **repeating group**, and 1NF rejects it for the same reason.



EVERY REQUEST IS CHECKED BEFORE IT IS ALLOWED TO TOUCH THE DATA

FIG. 2 — THE LAYERS OF A DBMS. USERS SEND A REQUEST TO THE QUERY PROCESSOR, THE QUERY PROCESSOR ASKS THE STORAGE MANAGER FOR THE BYTES, AND THE DATA DICTIONARY IS CONSULTED AT EVERY STEP SO THAT THE SYSTEM ALWAYS KNOWS THE SHAPE OF ITS OWN TABLES.

SECOND NORMAL FORM (2NF)

A table is in **2NF** when it is already in 1NF and it has **no partial dependency**. A partial dependency occurs when a non-key column depends on only **part** of a composite primary key.

A **composite key** is built from more than one column, and the trouble appears only then. If the key is `(Roll_No, Subject)` and `Subject_Name` depends only on the subject, the table is in 1NF but not in 2NF, because the subject name is stored again for every student taking it.

The cure is to move that column into a table of its own, keyed only by the subject; after the split the dependency is no longer partial. A table whose key is a **single** column cannot fail this test at all, which is why 2NF matters most in many-to-many relationships.

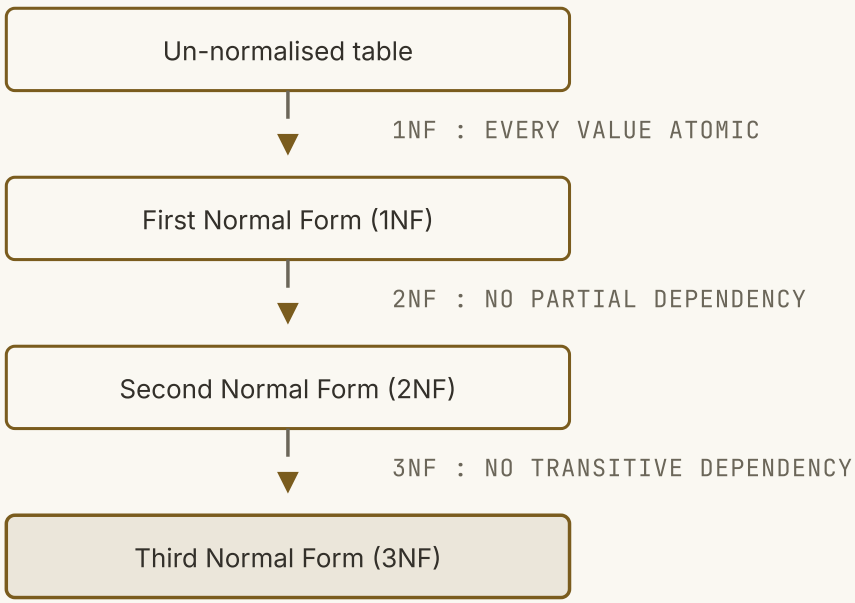


FIG. 1 — THE NORMALIZATION LADDER. EACH STEP REMOVES ONE KIND OF DEPENDENCY, SO THE TABLE BECOMES SMALLER, CLEANER AND FREE OF ANOMALIES.

THIRD NORMAL FORM (3NF)

A table is in **3NF** when no non-key column depends on another non-key column — a **transitive** dependency.

SQL PROGRAM

NORMAL_DEMO.SQL

```
-- STUDENT is NOT in 3NF: DEPT_NAME depends on DEPT_ID
CREATE TABLE STUDENT (
  ROLL_NO NUMBER(4) PRIMARY KEY,
  NAME    VARCHAR2(20),
  DEPT_ID NUMBER(3),
  DEPT_NAME VARCHAR2(20)
);
CREATE TABLE DEPARTMENT (
  DEPT_ID   NUMBER(3) PRIMARY KEY,
  DEPT_NAME VARCHAR2(20)
);

CREATE TABLE STUDENT (
  ROLL_NO NUMBER(4) PRIMARY KEY,
  NAME    VARCHAR2(20),
  DEPT_ID NUMBER(3),
  FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID)
);

SELECT S.NAME, D.DEPT_NAME
FROM   STUDENT S, DEPARTMENT D
WHERE  S.DEPT_ID = D.DEPT_ID;
```

Output

OUTPUT

```
NAME     DEPT_NAME
-------  ---------
Rahul    Computer
Anjali   Commerce
```

03 QUESTION THREE · KEYS

Explain Primary Key, Foreign Key & Unique Key with Example.

4 A4 PAGES
SQL EXAMPLE ON PAGE 4

WHAT IS A KEY? WHAT IS A PRIMARY KEY?

In a database table it is essential to identify every record uniquely. A **key** is that identifying column or set of columns. A **primary key** is a column, or a group of columns, that identifies each row **uniquely and completely**, and no row may repeat its value.

For example, in a student table `Roll_No` can be the primary key, because every student has a different roll number and it never changes. A primary key can never contain `NULL` or a duplicate, and it must be unique — that is the property called **entity integrity**.

A key may also be built from several columns; then it is called a **composite key**.

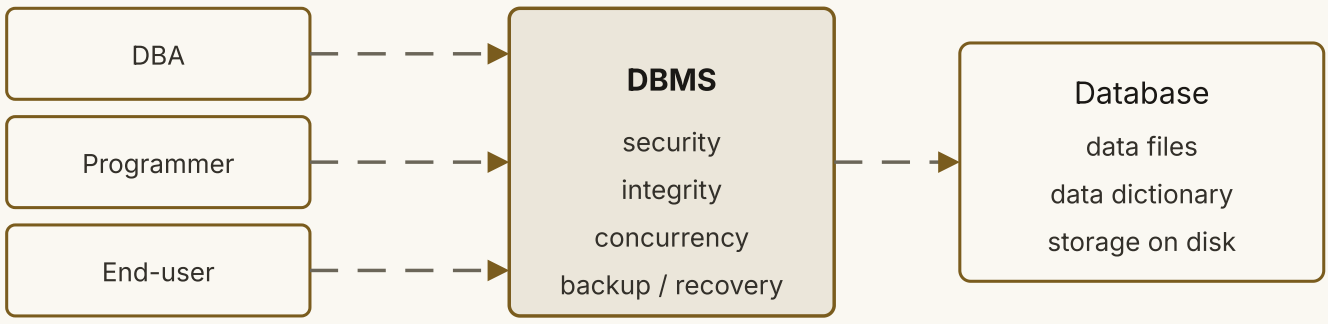


FIG. 1 — A DBMS DOES NOT SIT BESIDE THE DATA; IT STANDS **BETWEEN** THE USERS AND THE DATA, CHECKING EVERY REQUEST BEFORE IT IS ALLOWED TO TOUCH THE RECORDS.

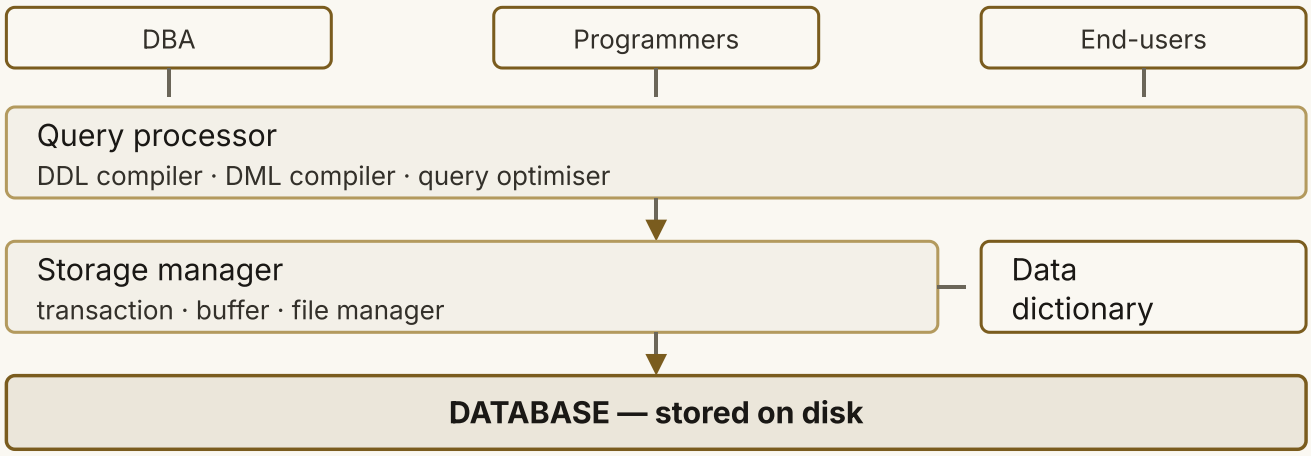
2. FOREIGN KEY

A **Foreign Key** is a column in one table that refers to the **primary key of another table**. Its job is to create a **relationship** between two tables and to enforce **referential integrity** — a value may exist in one table only if it already exists in the other. An order from an unknown customer simply cannot be entered.

3. UNIQUE KEY

A **Unique Key** is a column, or group of columns, whose values must never repeat. It guarantees that no two rows carry the same value, but unlike a primary key it may allow `NULL`. A table may have **only one** primary key but **many** unique keys.

4. Candidate and Composite Keys. A **candidate key** is any column, or set of columns, that could serve as the primary key because no value repeats; one is finally chosen as the primary key and the rest become unique keys. A **composite key** is made from two or more columns together.



EVERY REQUEST IS CHECKED BEFORE IT IS ALLOWED TO TOUCH THE DATA

FIG. 2 — THE LAYERS OF A DBMS. USERS SEND A REQUEST TO THE QUERY PROCESSOR, THE QUERY PROCESSOR ASKS THE STORAGE MANAGER FOR THE BYTES, AND THE DATA DICTIONARY IS CONSULTED AT EVERY STEP SO THAT THE SYSTEM ALWAYS KNOWS THE SHAPE OF ITS OWN TABLES.

DIFFERENCES BETWEEN PRIMARY, FOREIGN AND UNIQUE KEYS

POINT	PRIMARY KEY	FOREIGN KEY	UNIQUE KEY
Purpose	Identifies each row	Links two tables	Stops duplicates
How many	Only one per table	Any number	Any number
NULL	Never allowed	May be allowed	May be allowed
Refers to	Itself	Another table's key	Itself

All three keys work together: the primary key gives a row its identity, the unique key stops a second person sharing the same email, and the foreign key stops an order belonging to a customer who does not exist. Together they keep the data correct in every direction.

How the Keys Are Used Together

Consider a small shop with two tables.

`CUSTOMER` holds `Cust_Id` as its **primary key** and `Email` as a **unique key**, so no customer can be stored twice and no two customers can share one address.

`ORDERS` holds `Order_Id` as its **primary key**, and its `Cust_Id` is a **foreign key** that points at `CUSTOMER.Cust_Id`.

The three keys then work as a team: the **primary key** gives every row an identity that never changes, the **unique key** stops the same person appearing twice under different ids, and the **foreign key** stops a fact existing where its own reason has vanished.

SQL PROGRAM

KEYS_DEMO.SQL

```sql
CREATE TABLE CUSTOMER (
  CUST_ID  NUMBER(4) PRIMARY KEY,        -- primary key
  NAME     VARCHAR2(20) NOT NULL,
  EMAIL    VARCHAR2(30) UNIQUE           -- unique key
);

CREATE TABLE ORDERS (
  ORDER_ID NUMBER(4) PRIMARY KEY,
  CUST_ID  NUMBER(4),                     -- foreign key
  AMOUNT   NUMBER(8),
  FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID)
);

INSERT INTO CUSTOMER VALUES (101,'Rahul','rahul@mail.com');
INSERT INTO CUSTOMER VALUES (102,'Anjali','anjali@mail.com');
INSERT INTO ORDERS VALUES (1,101,2500);
INSERT INTO ORDERS VALUES (2,101,900);

SELECT C.NAME, O.AMOUNT
FROM   ORDERS O, CUSTOMER C
WHERE  O.CUST_ID = C.CUST_ID;
```

Output

OUTPUT

```
NAME    AMOUNT
------  ------
Rahul   2500
Rahul   900
```

CONCLUSION

An order for Cust_ID 999 would be refused, because that customer does not exist.

04 QUESTION FOUR · DBMS VS RDBMS

Differentiate between DBMS and RDBMS.

4 A4 PAGES
SQL JOIN EXAMPLE ON PAGE 4

DIFFERENCE BETWEEN DBMS AND RDBMS

An ordinary DBMS keeps data in separate files, and the links between those files — a customer with his orders, a student with his marks — are written by the programmer by hand. An RDBMS keeps data in **related tables** and lets the keys do the linking, so the same questions can be asked across many tables at once. This is the single change that turns a collection of files into a database.

POINT	DBMS	RDBMS
Storage	Files, one flat list	Related tables
Links	Written by the programmer	Through keys
Redundancy	Often repeats data	Each fact stored once
Query	Full scan of the file	SQL over many tables
Example	FoxPro, dBase	Oracle, MySQL, SQLite

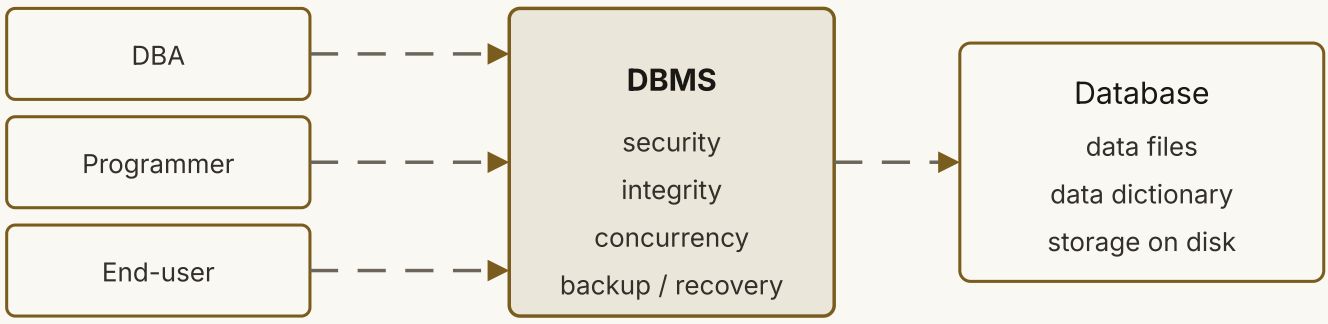


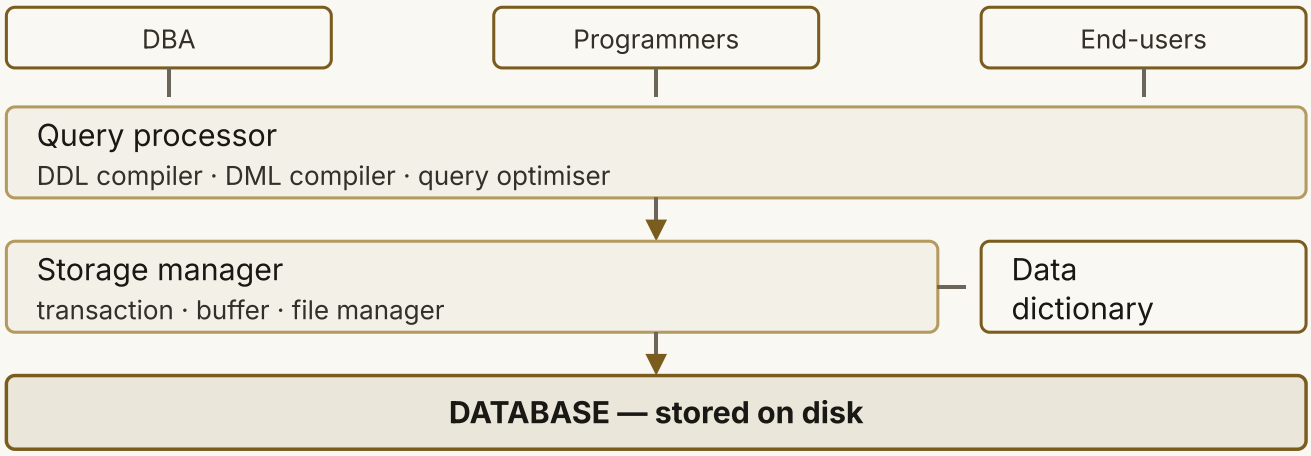
FIG. 1 — A DBMS DOES NOT SIT BESIDE THE DATA; IT STANDS **BETWEEN** THE USERS AND THE DATA, CHECKING EVERY REQUEST BEFORE IT IS ALLOWED TO TOUCH THE RECORDS.

WHAT IS AN RDBMS?

An **RDBMS (Relational Database Management System)** is a special form of DBMS in which the data is stored in **relations** — structured tables of rows and columns — and the tables are connected through common keys. It is based on the **relational model** of **E. F. Codd** and is queried using **SQL**.

Its greatest feature is **JOIN**, which combines related tables and returns the result as one answer, so information spread across many tables can be read together.

In a relation the rows are called **tuples** and the columns **attributes**. Two **integrity rules** hold in every relation: **entity integrity**, meaning no key may be `NULL`, and **referential integrity**, meaning a foreign key value must already exist as a key somewhere else.



EVERY REQUEST IS CHECKED BEFORE IT IS ALLOWED TO TOUCH THE DATA

FIG. 2 — THE LAYERS OF A DBMS. USERS SEND A REQUEST TO THE QUERY PROCESSOR, THE QUERY PROCESSOR ASKS THE STORAGE MANAGER FOR THE BYTES, AND THE DATA DICTIONARY IS CONSULTED AT EVERY STEP SO THAT THE SYSTEM ALWAYS KNOWS THE SHAPE OF ITS OWN TABLES.

ADVANTAGES OF AN RDBMS

Less redundancy: a fact is stored once, so the data stays consistent and the file becomes smaller.

Easy to update: one change is made in one place and every part of the system sees it immediately.

Easy to query: a question such as “name and phone of every customer with an order above 5000” is one SQL statement, though the answer is spread over three tables.

Reliability: an RDBMS supports the **ACID properties** — **Atomicity** (a transaction is all or nothing), **Consistency**, **Isolation** and **Durability** — so data remains safe even if the machine stops halfway through a change.

Security: permissions are given table by table, so every user sees exactly what he should see.

Examples are **Oracle, MySQL, Microsoft SQL Server, PostgreSQL** and **SQLite**. An RDBMS is best where the data is large and complex and where the relationships between tables really matter, which is why banks and large organisations insist on one.

Shared use: many users may work on the same data at once, each seeing a consistent view even while another is changing it. **Standard language:** almost every RDBMS understands **SQL**, so a query written for one runs on another with little change. **Recovery:** if the machine stops halfway through a transaction, the log on disk returns the system to the last completed state rather than a half-written one.

An RDBMS is best where the data is large and complex and where the relationships between tables really matter, which is why banks, airlines and large organisations insist on one.

SQL PROGRAM

JOIN_DEMO.SQL

```sql
-- three related tables
CREATE TABLE DEPT  (DEPT_ID NUMBER(2) PRIMARY KEY, DEPT_NAME VARCHAR2(15));
CREATE TABLE EMP   (EMP_ID  NUMBER(3) PRIMARY KEY,
                    NAME     VARCHAR2(15),
                    DEPT_ID  NUMBER(2),
                    SALARY   NUMBER(8),
                    FOREIGN KEY (DEPT_ID) REFERENCES DEPT(DEPT_ID));
CREATE TABLE BONUS (EMP_ID NUMBER(3) PRIMARY KEY, AMOUNT NUMBER(8));

INSERT INTO DEPT  VALUES (1,'Sales');
INSERT INTO DEPT  VALUES (2,'IT');
INSERT INTO EMP   VALUES (10,'Rahul', 1, 30000);
INSERT INTO EMP   VALUES (11,'Anjali',1, 35000);
INSERT INTO EMP   VALUES (12,'Vikas', 2, 40000);
INSERT INTO BONUS VALUES (10, 5000);
INSERT INTO BONUS VALUES (12, 7000);

-- JOIN across all three tables
SELECT D.DEPT_NAME, E.NAME, E.SALARY + B.AMOUNT AS TOTAL
FROM   EMP E
JOIN   DEPT  D ON E.DEPT_ID = D.DEPT_ID
JOIN   BONUS B ON E.EMP_ID  = B.EMP_ID;
```

Output

OUTPUT

```
DEPT_NAME  NAME     TOTAL
---------  ------  -----
Sales       Rahul   35000
IT          Vikas   47000
```

CONCLUSION

The JOIN is the real strength of an RDBMS: three tables, one answer.